Martín De Angulo Ulloa – 202421628

Pedro Pablo Muñoz - 202420736

Tecnología e Infraestructura de Computo

Caso 3 - Concurrencia y Sincronización de Procesos

Caso 3 - Concurrencia y Sincronizacion de Procesos 1
1. Introducción 2
2. Diseño del Sistema 2
2.1 Arquitectura General 2
2.2 Diagrama de Clases 3
2.3 Clase Buzon 3
2.4 Clase Monitor 4
3. Sincronizacion entre Actores 4
3.1 Tabla de Patrones de Espera 4
3.2 Espera Pasiva 4
3.3 Espera Semi-Activa 4
3.4 Sincronización por Preja de Actores 5
4. Terminacion del Sistema 6
5. Parámetros de Configuración 6
6. Validación 7
6.1 Prueba de Terminación Correcta 7
6.2 Prueba de Buzones Vacios 7
6.3 Prueba de Condiciones de Carrera 7
6.4 Prueba con Diferentes Configuraciones 8
7. Conclusiones 8

1. Introducción

El presente informe describe el diseño e implementación de un simulador concurrente de un sistema IoT (Internet of Things) para un campus universitario. El sistema coordina la generación de eventos desde múltiples sensores (productores), realiza detección de anomalías y encamina la información a diferentes servidores de consolidación.

El problema se planteo como un sistema productor-consumidor en cadena, donde cada actor es productor para el siguiente y consumidor del anterior. Los mecanismos de sincronización utilizados son exclusivamente las primitivas básicas de Java: synchronized, wait, notifyAll y yield.

2. Diseño del Sistema

2.1 Arquitectura General

El sistema sigue una cadena de procesamiento en la que participan 9 clases. El flujo de eventos es el siguiente:

- Sensores generan eventos y los depositan en el BuzonEntrada
- El Broker lee del BuzonEntrada, clasifica eventos como normales o anómalos
- Eventos anómalos van al BuzonAlertas; normales al BuzonClasificacion
- El Administrador lee del BuzonAlertas y reenvia inofensivos al BuzonClasificacion
- Los Clasificadores leen del BuzonClasificacion y enrutan al BuzonConsolidacion correcto
- Los Servidores leen de su BuzonConsolidacion y procesan cada evento

2.2 Diagrama de Clases

El sistema se compone de 9 clases principales:

Clase	Rol	Herencia
Evento	Objeto de datos transportado entre actores	Object
Monitor	Encapsula sincronización y conteo de clasificadores	Object
Buzon	Buffer genérico con 4 métodos de espera	Object
Sensor	Productor de eventos	Thread
Broker	Consumidor de entrada / Productor de alertas y clasificación	Thread
Administrador	Consumidor de alertas / Productor de clasificación	Runnable
Clasificador	Consumidor de clasificación / Productor de consolidación	Thread
Servidor	Consumidor final de consolidación	Thread
Main	Punto de entrada, crea e inicia todos los componentes	Object

2.3 Clase Buzon

La clase Buzon es el núcleo del sistema. Es un buffer genérico parametrizado que soporta capacidad ilimitada (valor -1) o limitada (valor N). Expone cuatro métodos según el patrón de espera requerido:

Metodo	Patron	Descripcion
depositar(e)	PASIVA (wait)	Espera si lleno. Usado por Clasificador → BuzonConsoidacion
depositarSemi(e)	SEMI-ACTIVA (wait(1) + yield)	Espera si lleno. Usado por Sensor, Broker, Administrador
retirar()	PASIVA (wait)	Espera si vacio. Usado por Broker, Clasificador, Servidor
retirarSemi()	SEMI-ACTIVA (wait(1) + yield)	Espera si vacío. Usado por Administrador
depositarPsiva(e)	PASIVA (wait)	Espera si lleno. Alias de depositar(), para mayor claridad

2.4 Clase Monitor

La clase Monitor tiene dos responsabilidades:

- Encapsula los métodos eserarPasiva() (wait) y esperarSemiActiva() (yield) usados por Buzon
- Lleva el conteo de clasificadores terminados mediante reportarTerminacion(), que es synchronized y retorna true cuando el ultimo clasificador termina.

3. Sincronizacion entre Actores

3.1 Tabla de Patrones de Espera

Segun el enunciado, los patrones de espera se asignan por relación entre actores:

Relacion	Metodo usado	Patron
Sensor → BuzonEtrada	depositarSemi()	SEMI-ACTIVA
BuzonEntrada → Broker	retirar()	PASIVA
Broker → BuzonAlertas	depositarSemi()	SEMI-ACTIVA
BuzonAlertas → Administrador	retirarSemi()	SEMI-ACTIVA
Broker → BuzonClasificacion	depositarSemi()	SEMI-ACTIVA
Administrador → BuzonClasificacion	depositar()	PASIVA
BuzonClasificacion → Clasificador	retirar()	PASIVA
Clasificador → BuzonConsolidacion	depositarPaiva()	PASIVA
BuzonCosolidacion → Servidor	retirar()	PASIVA

3.2 Espera Pasiva

La espera pasiva se implementa con wait() y notifyAll() dentro de metodos synchronized. El hilo que llama a wait() libera el lock del objeto y queda suspendido hasta que otro hilo llame a notifyAll(). Esto garantiza que el hilo no consume CPU mientras espera.

Ejemplo en retirar ():

```
1. while (cola.isEmpty())
2.     {try { wait(); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }}
```

3.3 Espera Semi-Activa

La espera semi-activa se implementa con wait(1) seguido de Thread.yield(). El wait(1) libera el lock por 1 milisegundo para que otros hilos puedan entrar al bloque sinchronized; luego yield()

cede la CPU al scheduler pero el hilo vuelve a competir en el siguiente ciclo. Sin el wait(1) inicial, el yield() dentro de synchronized no tendria efecto porque el lock no se liberaria.

Ejemplo en retirarSemi():

```
while (cola.isEmpty())
   {try { wait(1); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    Thread.yield();}
```

3.4 Sincronización por Preja de Actores

Sensor - BuzonEntrada

El Sensor llama a depositarSemi() que usa wait(1)+yield si el buzon esta lleno (aunque es ilimitado, nunca bloquea). El Broker llama a retirar() que hace wait() si esta vacio, y es notificado por notifyAll() cuando el Sensor deposita.

Broker - BuzonAlertas

El Broker deposita con depositarSemi() (semi-activa, buzon ilimitado). El Administrador retira con retirarSemi() que usa wait(1)+yield si esta vacio. El Broker notifica con notifyAll() al depositar.

Broker / Administrador - BuzonClasificacion

Buzon limitado (tam1). El Broker y el Administrador depositan con depositarSemi() / depositar() respectivamente, esperando si esta lleno. Los Clasificadores retiran con retirar() (wait) y notifican al retirar para despertar a los productores bloqueados.

Clasificador - BuzonConsolidacion

Buzon limitado (tam2), uno por servidor. El Clasificador deposita con depositarPasiva() (wait si lleno). El Servidor retira con retirar() (wait si vacio). Ambos llaman notifyAll() al completar su operacion.

Conteo del ultimo Clasificador - Monitor

Monitor.reportarTerminacion() es synchronized. Cada Clasificador lo llama al terminar; el que recibe true es el ultimo y ejecuta enviarFinServidores(), depositando un evento FIN en cada BuzonConsolidacion.

4. Terminacion del Sistema

La cadena de terminacion sigue el siguiente orden:

Actor	Condicion de terminacion	Accion antes de terminar
Sensor	Ha generado id * base eventos	Ninguna
Broker	Ha procesado totalEventos eventos	Deposita 1 evento FIN en BuzonAlertas
Administrador	Recibe evento FIN del Broker	Deposita nc eventos FIN en BuzonClasificacion
Clasificador	Recibe evento FIN del Administrador	El ultimo deposita ns FIN en BuzonConsolidacion
Servidor	Recibe evento FIN del ultimo Clasificador	Ninguna

El metodo main() llama join() sobre todos los hilos en el mismo orden de terminacion, garantizando que el sistema espera a que todos terminen antes de verificar los buzones.

5. Parámetros de Configuración

El sistema lee los parametros desde el archivo config.properties en formato clave=valor:

Parametro	Clave en archivo	Descripcion
ni	num_sensores	Numero de sensores IoT
base	num_eventos_base	Eventos base; sensor i genera base * i eventos
nc	num_clasificadores	Numero de clasificadores
ns	num_servidores	Numero de servidores de despliegue
tam1	capacidad_buzon_clasificacion	Capacidad del BuzonClasificacion
tam2	capacidad_buzon_consolidacion	Capacidad de cada BuzonConsolidacion

Ejemplo de archivo de configuracion:

```
num_sensores=3
num_eventos_base=2
num_clasificadores=2
num_servidores=2
capacidad_buzon_clasificacion=5
capacidad_buzon_consolidacion=4
```

6. Validación

6.1 Prueba de Terminación Correcta

Se ejecuto el sistema con la configuración de ejemplo (3 sensores, base=2, 2 clasificadores, 2 servidores, tam1=5, tam2=4). El total de eventos esperados es 2+4+6=12.

Resultado verificado en la salida:

- Todos los sensores terminaron tras generar sus eventos asignados
- El Broker proceso exactamente 12 eventos
- El Administrador recibio el evento FIN y envio 2 FIN a los clasificadores
- El ultimo clasificador envio los FIN a ambos servidores
- Todos los servidores terminaron
- Verificacion final: todos los buzones quedaron vacios

6.2 Prueba de Buzones Vacios

Al finalizar la ejecucion, Main verifica explicitamente que cada buzon este vacio con el metodo estaVacio(). En todas las pruebas realizadas, la salida confirmo:

- Buzon de entrada vacio
- Buzon de Alertas vacio
- Buzon de clasificacion vacio
- Buzon de Consolidacion No.1 Vacio
- Buzon de Consolidacion No.2 Vacio

6.3 Prueba de Condiciones de Carrera

El metodo Monitor.reportarTerminacion() es synchronized, lo que garantiza que el incremento del contador y la verificacion del ultimo clasificador ocurren atomicamente. Se verifico con multiples ejecuciones que siempre hay exactamente un mensaje '*** Clasificador es el ULTIMO en terminar ***' y que los eventos FIN a los servidores se envian exactamente una vez.

6.4 Prueba con Diferentes Configuraciones

Se probaron configuraciones adicionales para verificar la generalización:

ni	base	nc	ns	tam1	tam2	Total eventos	Resultado
3	2	2	2	5	4	12	OK - tdos vacios
2	3	1	3	8	6	9	OK - todos vacios
4	1	3	2	3	2	10	OK - todos vacios

7. Conclusiones

- El patron productor-consumidor en cadena permite modelar naturalmente el flujo de eventos del sistema IoT, donde cada actor consume del buzón anterior y produce para el siguiente.
- La clase Buzon unifica el manejo de todos los buffers del sistema. Los cuatro métodos (depositar, depositarSemi, retirar, retirarSemi) permiten aplicar el patrón de espera correcto segun la relacion entre actores, sin duplicar logica.
- La diferencia entre espera pasiva (wait/notifyAll) y semi-activa (wait(1)+yield) es conceptualmente importante: la pasiva suspende completamente el hilo hasta ser notificado, mientras la semi-activa cede la CPU pero vuelve a competir periodicamente.
- El uso de Monitor.reportarTerminacion() con synchronized resuelve de forma atomica el problema de detectar el ultimo clasificador, evitando condiciones de carrera en ese punto critico.
- La cadena de eventos FIN garantiza una terminación ordenada y controlada de todos los actores, asegurando que ningún buzón quede con eventos sin procesar.